4. Reglas SDT y tabla de símbolos

Alcance del análisis semántico

Las reglas SDT se asocian a las producciones de la gramática. Se evalúan sobre el árbol sintáctico después de que el parser acepta Program. Revisan declaraciones, uso de identificadores, ámbitos y compatibilidad de tipos; no ejecutan el programa.

Cada nodo representa un símbolo gramatical y conserva su producción, sus hijos y su ubicación. Las hojas mantienen el lexema y la posición del token. Esto permite relacionar los errores semánticos con la parte del programa que los originó.

Un error léxico impide continuar con el análisis sintáctico normal. Un error sintáctico impide evaluar SDT sobre un árbol completo. Si el parser acepta y SDT detecta un fallo, ambos resultados se distinguen.

Atributos y tabla de símbolos

- Expresiones: `data_type`; `literal_lexeme` solo para literales simples, sin calcular valores.
- Inicialización: `present` y tipo de la expresión. Bloques: `scope_id`. Sentencias revisadas: `valid`.
- Símbolo: nombre, tipo, ID de ámbito, línea y columna de declaración, presencia de inicializador, tipo y lexema inicial cuando corresponde.
- Cada bloque define un ámbito. La búsqueda de identificadores va del interno al externo. Los duplicados se rechazan en el mismo ámbito; se permite sombreado en uno interior. Al terminar el bloque, su ámbito deja de estar activo.
- Insertar la variable antes de revisar su inicializador. Se comprueban nombres y tipos, sin análisis de inicialización definida. `int x=x;` no se rechaza por esa causa.

Política de tipos del subconjunto

`INT, FLOAT, CHAR, STRING` producen tipos `int, float, char, string`. Los únicos tipos declarables son `int, float, char`. Los literales deben llegar validados por el lexer; SDT no los vuelve a interpretar.

Para inicialización, asignación y retorno se admite identidad o ampliación `char -> int -> float`. Se rechazan conversiones de estrechamiento. Esta es una política simplificada del proyecto; no reproduce todas las conversiones de ISO C.

Destino / origen	char	int	float
char	Sí	No	No
int	Sí	Sí	No
float	Sí	Sí	Sí

Las cadenas no son asignables a ningún tipo declarable. En aritmética se promueve `char` a `int`; si hay `float`, el resultado es `float`. Las comparaciones y `!` producen `int`. El módulo `%` solo admite enteros o caracteres y produce `int`. Las condiciones deben ser numéricas; `return` debe ser compatible con `int`.

Acciones por producción

ID(s)	Producción o familia	Acción semántica
0	`Start -> Program`	El parser reconoce el ítem aumentado y emite ACCEPT. Program es la raíz del árbol; no corresponde una reducción ordinaria por 0.
1	`Program -> int main ( ) Block`	Fijar return_type=int; visitar el bloque de main. Atributo valid=true solo si termina sin fallo.
2	`Block -> { Items }`	Asignar un scope_id nuevo; revisar Items en ese ámbito y cerrarlo al salir del bloque.
3	`Items -> Items Item`	Visitar de izquierda a derecha: primero la lista previa y después el nuevo elemento.
4	`Items -> epsilon`	No modificar los símbolos. La lista vacía es válida.
5, 6	`Item -> Decl | Stmt`	Visitar el único hijo y propagar su validez.
7	`Decl -> Type ID Init ;`	Obtener Type.data_type, rechazar duplicados del ámbito actual, insertar ID, revisar Init y comprobar assignable(destino,origen).
8, 9, 10	`Type -> int | float | char`	Sintetizar data_type con el tipo indicado por la palabra clave.
11	`Init -> = Expr`	Sintetizar present=true y data_type=Expr.data_type. El padre Decl valida la compatibilidad.
12	`Init -> epsilon`	Sintetizar present=false. No exige expresión ni tipo de inicializador.
13, 14	`Stmt -> Matched | Unmatched`	Visitar el hijo; no crear ámbito por la sola existencia de Stmt.
15, 16	`Matched -> Simple | Block`	Visitar el hijo. Si es Block, él abre y cierra su ámbito.
17, 20	`if ( Expr ) ... else ...`	Exigir condición numérica; visitar ambos cuerpos en orden fuente para comprobar estáticamente sus nombres y tipos.
18, 21	`while ( Expr ) ...`	Exigir condición numérica; revisar el cuerpo una vez, sin ejecutar iteraciones.
19	`if ( Expr ) Stmt`	Exigir condición numérica y visitar Stmt.
22	`Simple -> ID = Expr ;`	Buscar ID como destino de escritura, obtener tipo de Expr y comprobar compatibilidad. No actualizar valores de ejecución.
23	`Simple -> return Expr ;`	Obtener tipo de Expr y comprobar que sea asignable a int, el tipo de retorno de main.
24	`Simple -> ;`	Aceptar la sentencia vacía sin alterar la tabla.
25, 28, 33, 36, 40, 44	`Expresiones de paso`	Propagar data_type y, si todavía es un literal simple, literal_lexeme del único hijo.
26, 27, 29, 30, 31, 32	`Igualdad y relaciones`	Exigir dos operandos numéricos y sintetizar int. El tipo del resultado no depende de calcular su valor.
34, 35, 37, 38	`Suma, resta, producto, división`	Exigir números; promover char a int. Si algún operando es float, el resultado es float; de otro modo, int.
39	`Mul -> Mul % Unary`	Exigir operandos char o int. Rechazar float y sintetizar int.
41, 42	`Unary -> + Unary | - Unary`	Exigir número y devolver promote(tipo del hijo).
43	`Unary -> ! Unary`	Exigir número y sintetizar int.
45	`Primary -> ID`	Buscar el nombre visible, sintetizar su tipo y registrar el vínculo con scope_id y línea de declaración.
46, 47, 48, 49	`Primary -> INT | FLOAT | CHAR | STRING`	Sintetizar int, float, char o string respectivamente. Conservar el lexema original del literal, sin evaluarlo.
50	`Primary -> ( Expr )`	Propagar los atributos de Expr; los paréntesis no crean un ámbito.

Ejemplos de compatibilidad y errores

Cada fila representa un cuerpo dentro de `int main(){ ... }`. Todas tienen sintaxis válida; la diferencia está en SDT.

Cuerpo	Resultado semántico
`int x=10; return x;`	Válido.
`int x="hello";`	SEM004: string no es asignable a int.
`char c='a';`	Válido: char a char.
`char c=65;`	SEM004: int a char es estrechamiento.
`char c='a'+1;`	SEM004: la suma promueve char a int y el destino es char.
`char c='a'; c=c+1;`	SEM003: la suma da int y la asignación a char se rechaza.
`char c='a'; int x=c+1;`	Válido: resultado int y destino int.
`int x; int x;`	SEM001: declaración duplicada en el mismo ámbito.
`y=1;`	SEM002: identificador no declarado.

El resultado registra el primer error semántico detectado. La línea, columna y lexema provienen del token responsable. Las ramas de if y el cuerpo de while se revisan estáticamente sin ejecutar el programa.

Ejemplo: del árbol a la tabla de símbolos

```
int main() {
    int x = 10;
    float y = x + 2.5;
    return x;
}
```

1. Program fija el tipo de retorno de main en int. El bloque abre el ámbito 0.
2. La primera declaración obtiene Type=int, registra x y revisa el literal 10. Su tipo es int: la inicialización es compatible.
3. La segunda declaración registra y como float. Para `x + 2.5`, la búsqueda de x devuelve int y el literal tiene tipo float. La suma sintetiza float; coincide con el tipo de y.
4. `return x;` busca x y obtiene int, compatible con el retorno de main. Al terminar el bloque se cierra su ámbito.

Nombre	Tipo	Ámbito	Línea	Información del inicializador
x	int	0	2	Presente; tipo int; lexema literal `10`.
y	float	0	3	Presente; tipo float; sin lexema literal único, porque es una expresión.

El análisis conserva los registros para mostrarlos en el resultado, aunque el ámbito deje de estar activo. No calcula el valor de y ni ejecuta el retorno. Las columnas proceden de los tokens originales, no del formato de este ejemplo.

Ejemplo de ámbitos y sombreado

```
int main() {
    int x = 1;
    { float x = 2.5; }
    return x;
}
```

La x interior pertenece a otro ámbito y puede tener otro tipo. Al cerrar sus llaves, `return x;` vuelve a encontrar la x exterior de tipo int. Dos declaraciones de x dentro del *mismo* bloque sí generan SEM001. Una variable declarada únicamente dentro de un bloque no queda visible fuera de él.

Sintaxis aceptada y semántica rechazada

En `int x="hello";`, el parser reconoce la producción de declaración. SDT obtiene tipo string para la expresión y tipo int para el destino; la matriz de compatibilidad rechaza la inicialización. El resultado conserva `Parsing Success!` y añade el error SDT con la posición de x. Un punto y coma faltante, en cambio, detiene el parser antes de esta revisión.

Resultados del análisis

```
Parsing Success!
SDT Verified!
```

Si hay error semántico, conservar `Parsing Success!` y emitir `SDT error in line n`. Con error de sintaxis emitir `Parsing error in line n`; con error léxico, `Lexical error in line n`. No informar SDT como verificado si una fase anterior falló.